

Practical-4

0/1 Knapsack Using Dynamic Programming

Theory :-

The 0/1 Knapsack problem is a Dynamic Programming problem in which we have a set of items, and each item has a weight and a value (profit). The objective is to select items such that the total weight does not exceed the knapsack capacity and the total profit is maximum.

In the 0/1 Knapsack, each item can either be:

- Selected (1), or
- Not selected (0)

An item cannot be selected more than once.

Code :-

```
#include <iostream>
using namespace std;

// Knapsack function
int knapsack(int capacity, int weight[], int value[], int n) {
    int dp[n + 1][capacity + 1];

    // Initialize DP table
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {

            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            }
            else if (weight[i - 1] <= w) {
                dp[i][w] = max(
                    value[i - 1] + dp[i - 1][w - weight[i - 1]],
                    dp[i - 1][w]
                );
            }
            else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
}
```

```

        return dp[n][capacity];
    }

// Main function
int main() {
    int n, capacity;

    cout << "Enter number of items: ";
    cin >> n;

    int weight[n], value[n];

    cout << "Enter weights of sack :\n";
    for (int i = 0; i < n; i++) {
        cin >> weight[i];
    }

    cout << "Enter profits corresponding to their sack weight :\n";
    for (int i = 0; i < n; i++) {
        cin >> value[i];
    }

    cout << "Enter knapsack capacity: ";
    cin >> capacity;

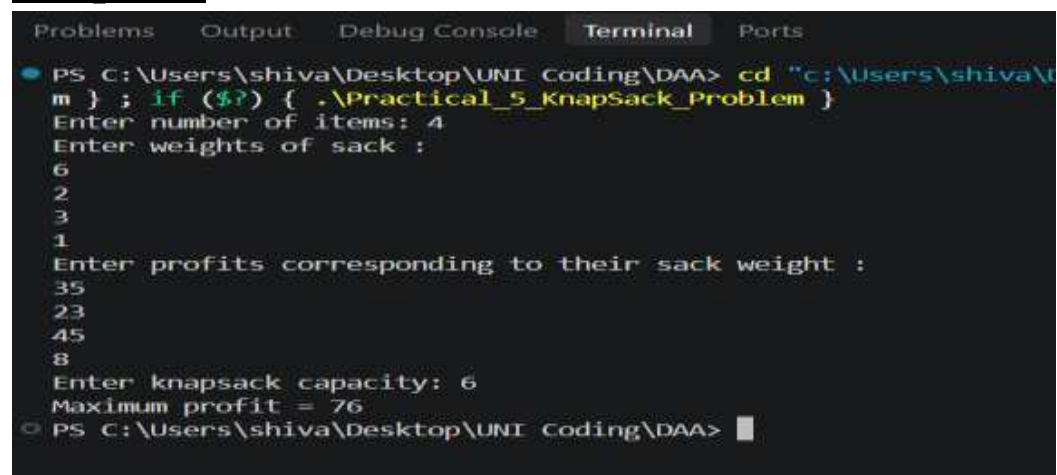
    int result = knapsack(capacity, weight, value, n);

    cout << "Maximum profit = " << result << endl;

    return 0;
}

```

Output :-



```

Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\shiva\Desktop\UNI Coding\DAA> cd "c:\Users\shiva\UNI Coding\DAA"
m } ; if ($?) { .\Practical_5_KnapSack_Problem }
Enter number of items: 4
Enter weights of sack :
6
2
3
1
Enter profits corresponding to their sack weight :
35
23
45
8
Enter knapsack capacity: 6
Maximum profit = 76
PS C:\Users\shiva\Desktop\UNI Coding\DAA>

```

Analysis Of Algorithm :- 0/1 Knapsack

- **Input:** Number of items, weights, values, and knapsack capacity.
- **Output:** Maximum possible profit.
- **Technique:** Dynamic Programming.
- **Time Complexity:** $O(n \times W)$
- **Space Complexity:** $O(n \times W)$

Where:

- n = number of items
- W = knapsack capacity